MOVIE RECOMMENDATION SYSTEM

A content-based recommender using bag-of-words features and cosine similarity on a curated 110-movie dataset

A Project Report Submitted in Partial Fulfilment of the Requirements for the

3rd Semester Evaluation of B.Tech (Computer Science & Engineering)

Subject: Machine Learning

Submitted by (Group – __)

Name	Roll No.
Shakti Subham Mishra	1ETBTA20251040
Apurba Mohanty	1ETBTA20251039
Preksha Singh	1ETBTA20251038
Puravi Dash	1ETBTA20251041

Project Guide: Ansh Agrawal

Department of Computer Science & Engineering

Faculty of Engineering & Technology, Sri Sri University

Academic Year 2026 – 2027

DECLARATION

We, the undersigned members of Group-__ (3rd Semester, B.Tech CSE), hereby declare that the project report entitled "Movie Recommendation System" submitted for the Machine Learning evaluation under the guidance of Ansh Agrawal (Project Guide) is an original and authentic record of the work carried out by us. The matter presented in this report has not been submitted, in part or in full, to any other university, institute, or examining body for the award of any degree or diploma.

We further declare that all sources of information, ideas, algorithms, and datasets used in the preparation of this report have been duly acknowledged and cited in the References section, and that this work does not, to the best of our knowledge, infringe upon any existing copyright.

Team Members:

- Shakti Subham Mishra (1ETBTA20251040)
- Apurba Mohanty (1ETBTA20251039)
- Preksha Singh (1ETBTA20251038)
- Puravi Dash (1ETBTA20251041)

Place: _____
Date: _____

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to everyone who supported us throughout the completion of this project. We are deeply thankful to our project guide, Ansh Agrawal, for their invaluable guidance, technical insights, and constant encouragement, which helped us turn a simple idea into a working and well-evaluated system.

We would also like to thank the faculty members of the Department of Computer Science & Engineering, Sri Sri University, for providing us with the necessary academic support, laboratory resources, and a conducive environment for learning and experimentation.

Finally, we would like to acknowledge the effort and cooperation of every member of our team, whose collective contribution, discussions, and dedication made this project possible.

ABSTRACT

Every streaming platform now offers thousands of movies, and viewers often spend more time choosing what to watch than watching it. Recommender systems solve this "choice overload" by suggesting titles that match a viewer's taste. This project builds a movie recommender that, given one movie a user liked, suggests the five most similar movies.

We use content-based filtering, the simplest family of recommender systems that needs no user ratings. Each movie is described by a short "tag" text built from its genres, director, two lead actors and a few plot keywords. The tags are turned into word-count vectors (bag of words), and cosine similarity between the vectors measures how alike two movies are. The model has no training loop, answers in well under a second, and every recommendation can be explained by the words two movies share.

We curated a dataset of 110 well-known movies (1968–2022), mixing Hollywood films, Indian films and Japanese animation, and covering 20 genres and 64 directors. Pre-processing merges multi-word names into single tokens and joins all attributes into one tag string, giving a 537-word vocabulary. The system is written in Python with pandas and scikit-learn. It is evaluated on a hand-labelled test set of 20 query movies with 26 expected recommendations, using Precision, Recall, F1-score and Hit Rate@5, and compared against a random baseline and a genre-only baseline.

The proposed model found all 26 expected movies (Recall = 1.00, Hit Rate@5 = 1.00, F1 = 0.41), compared with a recall of 0.77 for the genre-only model and 0.05 for random picks; its precision of 0.26 is the highest possible on this test set. The results show that a very simple, explainable model can give useful recommendations and is a solid base for a larger hybrid system.

TABLE OF CONTENTS

1. INTRODUCTION 7
1.1 Background 7
1.2 Motivation 7
1.3 Problem Statement 7
1.4 Objectives 7
1.5 Scope 7
2. LITERATURE REVIEW & EXISTING SYSTEM 8
2.1 Existing Methods 8
2.2 Deep-Learning-Based Approaches 8
2.3 Limitations of Existing Systems 8
2.4 Proposed Improvement 8
3. PROPOSED SYSTEM / METHODOLOGY 9
3.1 System Overview 9
3.2 Workflow 9
3.3 Dataset 9
3.4 Preprocessing 10
3.5 Algorithm / Model 10
4. SYSTEM DESIGN & IMPLEMENTATION 11
4.1 Architecture Details 11
4.2 Technologies Used 11
4.3 Implementation Steps 11
4.4 Important Code Snippet 11
5. EXPERIMENTAL SETUP & RESULTS 13
5.1 Dataset Details 13
5.2 Model Configuration 13
5.3 Evaluation Metrics 13
5.4 Results 13
6. RESULTS ANALYSIS & DISCUSSION 15
6.1 Confusion Matrix 15
6.2 Predictions and Recommendation Quality 15
6.3 Comparison with Baseline 15
6.4 Observations 16
6.5 Strengths 16

6.6 Limitations 16
7. CONCLUSION, FUTURE SCOPE & REFERENCES 17
7.1 Conclusion 17
7.2 Future Scope 17
7.3 Limitations Recap 17
7.4 References 17

1. INTRODUCTION

1.1 Background

Recommender systems are machine-learning systems that predict which items a user is likely to enjoy. They power the "Because you watched…" rows on Netflix, product suggestions on Amazon and the video feed on YouTube. In the movie domain the catalogue is far larger than any person can browse, which makes recommendation one of the most widely used everyday applications of machine learning.

1.2 Motivation

Search only helps when users already know what they want; while browsing, the platform has to suggest something, and poor suggestions lead to wasted time and lost users. Most tutorials jump straight to matrix-factorisation or deep-learning models that need millions of ratings and powerful hardware. We wanted to show that a model a student can explain line by line, built on a small dataset, can already give sensible recommendations, and to measure honestly how good it is.

1.3 Problem Statement

Given the title of one movie that a user liked, the system returns the five most similar movies from its catalogue, ranked by similarity, using only descriptive information about the movies (genres, director, lead cast and plot keywords).

1.4 Objectives

1. Curate a clean dataset of 110 popular movies with genres, director, lead cast and plot keywords.
2. Design and implement a content-based recommendation model using bag-of-words vectors and cosine similarity.
3. Make the system easy to use, including tolerance to spelling mistakes in the movie title.
4. Evaluate the model with Precision, Recall, F1-score and Hit Rate@5 on a labelled test set.
5. Compare the model against a random baseline and a genre-only baseline.

1.5 Scope

The project covers content-based recommendations for a fixed catalogue of 110 movies, a command-line program in Python and an offline evaluation. User ratings and watch history, personalised profiles built from many movies, deep-learning models, and a web or mobile front end are explicitly out of scope; they are discussed as future work in Chapter 7.

2. LITERATURE REVIEW & EXISTING SYSTEM

2.1 Existing Methods

Recommender systems are usually grouped into three families [1]. Popularity-based systems recommend the most-watched or highest-rated items; they are trivial to build but give every user the same list. Content-based filtering represents each item by its features and recommends items similar to those the user liked [2]; classic implementations use word-count or TF-IDF vectors with cosine similarity, a technique borrowed from information retrieval [3]. Collaborative filtering ignores item content and uses the user–item rating matrix instead: neighbourhood methods recommend items liked by similar users, and matrix-factorisation methods, popularised by the Netflix Prize, learn hidden factors for users and items [4].

2.2 Deep-Learning-Based Approaches

Recent systems use neural networks. Neural Collaborative Filtering replaces the dot product of matrix factorisation with a multi-layer perceptron [5], and YouTube's recommender uses deep networks for candidate generation and ranking over billions of interactions [6]. Text and image encoders, such as sentence embeddings of plot summaries or CNN features of posters, are also used to build richer content representations. These models reach state-of-the-art accuracy on large benchmarks such as MovieLens [7], but they need large datasets, GPUs and careful tuning.

2.3 Limitations of Existing Systems

- Collaborative filtering needs a large rating history and cannot recommend a new movie that nobody has rated yet (the cold-start problem).
- Deep-learning recommenders need large datasets, GPUs and tuning, and act as black boxes whose suggestions are hard to explain.
- Popularity lists are not personalised, and simple genre matching is too coarse: every action-crime film looks the same to it.

2.4 Proposed Improvement

Our system uses content-based filtering, so it needs no ratings and works for a brand-new movie as soon as its description is added. Instead of matching genres alone, it combines genres with the director, lead actors and plot keywords, which lets it recognise sequels, shared universes and a director's style. The whole model is one similarity table, so it is built instantly on a laptop, and each recommendation can be explained by listing the words two movies share.

3. PROPOSED SYSTEM / METHODOLOGY

3.1 System Overview

The system reads the movie catalogue from a CSV file, builds one "tag" text per movie, converts the tags into word-count vectors and pre-computes the cosine similarity between every pair of movies. At query time the user's title is matched to a catalogue entry (tolerating typos), the matching row of the similarity table is sorted, and the five most similar movies are returned with a match percentage. Figure 1 shows the pipeline.



Figure 1: End-to-end pipeline of the movie recommendation system

3.2 Workflow

1. **Input:** the title of a movie the user liked, typed on the command line, together with the catalogue file movies.csv.

2. **Preprocessing:** director and actor names are merged into single lowercase tokens (e.g. "Christopher Nolan" → christophernolan), and genres, director, cast and keywords are joined into one tag string per movie.

3. **Feature Extraction / Core Processing:** CountVectorizer converts each tag string into a 537-dimensional word-count vector (bag of words).

4. **Model / Decision Step:** cosine similarity is computed between the query movie's vector and every other movie's vector, and movies are ranked by this score.

5. **Output:** the five highest-scoring movies, each with its similarity shown as a match percentage.

3.3 Dataset

Because the project had to run fully offline, the team curated its own dataset, movies.csv, of 110 well-known movies released between 1968 and 2022. It mixes Hollywood blockbusters and classics, Indian films (Hindi and Telugu) and Japanese animation, and deliberately includes several franchises (Batman, Marvel, The Lord of the Rings, Harry Potter, Toy Story) so that similarity can be checked. Each row has six fields: title, year, genres (pipe-separated), director, two lead actors and four to six plot keywords.

The catalogue covers 20 genres and 64 directors. Genres overlap, so there is no single class balance; the most frequent genres are Drama (46 movies), Action (32), Adventure (31) and Sci-Fi (29). Genres, directors and cast were taken from public film information, and the keywords were written by the team.

For evaluation, the team also labelled 20 query movies with 26 "expected" recommendations (Section 5.1).

3.4 Preprocessing

- **Name joining:** spaces, dots and hyphens are removed from director and actor names and the result is lowercased, so each person becomes one token and two people who share a first name are not confused.
- **Tag construction:** genres, director, cast and keywords are joined into one text field, tags. For example, The Dark Knight becomes "Action Crime Drama christophernolan christianbale heathledger batman joker gotham vigilante superhero".
- **Vectorisation:** CountVectorizer lowercases and tokenises the tags and builds a 537-word vocabulary; each movie becomes a sparse vector of word counts. No data augmentation is needed.

3.5 Algorithm / Model

The model is content-based filtering with cosine similarity. Each movie is a vector of word counts, and the similarity of two movies A and B is $\cos(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$, which is 1 when their tags are identical and 0 when they share no words. Because each word normally appears only once in a movie's tags, this equals the number of shared words divided by the square root of the product of the two tag lengths. For example, The Dark Knight (11 words) and Batman Begins (10 words) share 7 words, so their similarity is $7 / \sqrt{110} \approx 0.67$.

All pairwise similarities are stored in a 110 × 110 matrix. To recommend, the system takes the row of the query movie, sorts it from highest to lowest, skips the movie itself and returns the top five. There are no layers, loss function or iterative training: fitting the model simply means building the vocabulary and the similarity matrix. This design was chosen because it is the simplest model that still captures sequels, shared universes and a director's style, it needs no user data, and every recommendation can be explained.

4. SYSTEM DESIGN & IMPLEMENTATION

4.1 Architecture Details

The system has five stages, summarised in Table 1. The pipeline diagram is shown in Figure 1.

Stage	Component	Output	Details
1. Data loading	pandas.read_csv	110 × 6 table	movies.csv
2. Tag builder	one_token() + string join	110 tag strings	genres + director + cast + keywords
3. Vectoriser	CountVectorizer	110 × 537 sparse matrix	default tokeniser, lowercase
4. Similarity	cosine_similarity	110 × 110 matrix	scores in [0, 1]
5. Recommender	find_title() + sort	Top-5 titles + scores	difflib cutoff 0.5, k = 5

4.2 Technologies Used

- Python 3 (tested on Python 3.14) with scikit-learn for CountVectorizer and cosine_similarity.
- pandas for loading the CSV dataset, NumPy (through scikit-learn) for the similarity matrix, and difflib from the Python standard library for typo-tolerant title matching.
- python-pptx native charts for the results figure and the presentation slides.
- Visual Studio Code and the Windows command line; the same script also runs unchanged in Jupyter Notebook or Google Colab.

4.3 Implementation Steps

1. **Data collection:** curated 110 movies with six attributes each into movies.csv and checked them for consistency.
2. **Preprocessing:** implemented one_token() and the tag builder in load_movies().
3. **Model:** implemented build_similarity() with CountVectorizer and cosine_similarity.
4. **Recommendation:** implemented find_title() (difflib, for typos) and recommend(), which ranks movies and returns the top five.
5. **Baselines:** implemented a random recommender (averaged over 100 runs) and a genre-only recommender that uses the same code on the genre column only.
6. **Evaluation:** built a 20-query labelled test set and computed the confusion matrix, Precision, Recall, F1 and Hit Rate@5 for all three models (python movie_recommender.py --evaluate).

4.4 Important Code Snippet

Listing 1: the core of the model (movie_recommender.py).

```
def build_similarity(df, column="tags"):
    vectors = CountVectorizer().fit_transform(df[column])
    return cosine_similarity(vectors)          # 110 x 110 matrix


def recommend(query, df, sim, k=5):
    title = find_title(query, df)              # typo-tolerant lookup
    i = df.index[df["title"] == title][0]
```

```
    ranked = sorted(enumerate(sim[i]), key=lambda pair: -pair[1])
    return title, [(df["title"][j], float(score))
                   for j, score in ranked if j != i][:k]
```

Listing 2: example run from the command line.

```
> python movie_recommender.py "The Dark Knight"
Because you liked 'The Dark Knight', you may also like:
  1. Batman Begins                 match 67%
  2. The Dark Knight Rises         match 60%
  3. Joker                         match 36%
  4. The Prestige                  match 29%
  5. The Godfather                 match 20%
```

5. EXPERIMENTAL SETUP & RESULTS

5.1 Dataset Details

The final catalogue has 110 movies. Content-based filtering has no training/validation split in the usual sense: the "model" is the similarity matrix built from the descriptions of all movies, and it uses no labels. To measure quality, the team built a separate labelled test set of 20 query movies. For each query we listed the one or two movies a fan would clearly expect to see: sequels (Terminator 2 → The Terminator), shared universes (Iron Man → The Avengers) or the same director and star (3 Idiots → PK). This gives 26 expected pairs. The labels were fixed before the evaluation was run and are never used to build the model. Every query has 109 candidate movies, so 20 × 109 = 2,180 decisions are evaluated.

5.2 Model Configuration

Parameter	Value
Input features	Genres + director + 2 lead actors + 4–6 keywords
Vectoriser	CountVectorizer (bag of words), 537 words
Similarity measure	Cosine similarity (110 × 110 matrix)
Training / loss	None, no iterative training
Recommendations per query (k)	5
Hardware / runtime	Laptop CPU; full evaluation runs in under 2 seconds

5.3 Evaluation Metrics

For each query, the five movies shown are positive predictions and all other movies are negative predictions; an expected movie is an actual positive. Summing over the 20 queries gives a confusion matrix (TP, FP, FN, TN), from which we compute:

Precision = TP / (TP + FP): the share of shown recommendations that were expected. The maximum possible here is 26 / 100 = 0.26, because only 26 expected answers exist for 100 recommendation slots.

Recall = TP / (TP + FN): the share of expected movies that were found. This is the most important metric for our task.

F1-score: the harmonic mean of precision and recall.

Accuracy = (TP + TN) / total: reported for completeness, but misleading here because about 95% of all decisions are true negatives.

Hit Rate@5: the share of queries with at least one expected movie in the top five, which is what a user would notice.

5.4 Results

Model	Accuracy	Precision	Recall	F1-Score
Random baseline (mean of 100 runs)	0.943	0.013	0.048	0.020
Genre-only model	0.961	0.200	0.769	0.317
Proposed model (full tags)	0.966	0.260	1.000	0.413

Hit Rate@5 was 0.06 for random picks, 0.80 for the genre-only model and 1.00 for the proposed model (Figure 2). Because the model has no iterative training, there are no training or validation curves and no risk of overfitting to a training set; the complete evaluation runs in under two seconds on a laptop.

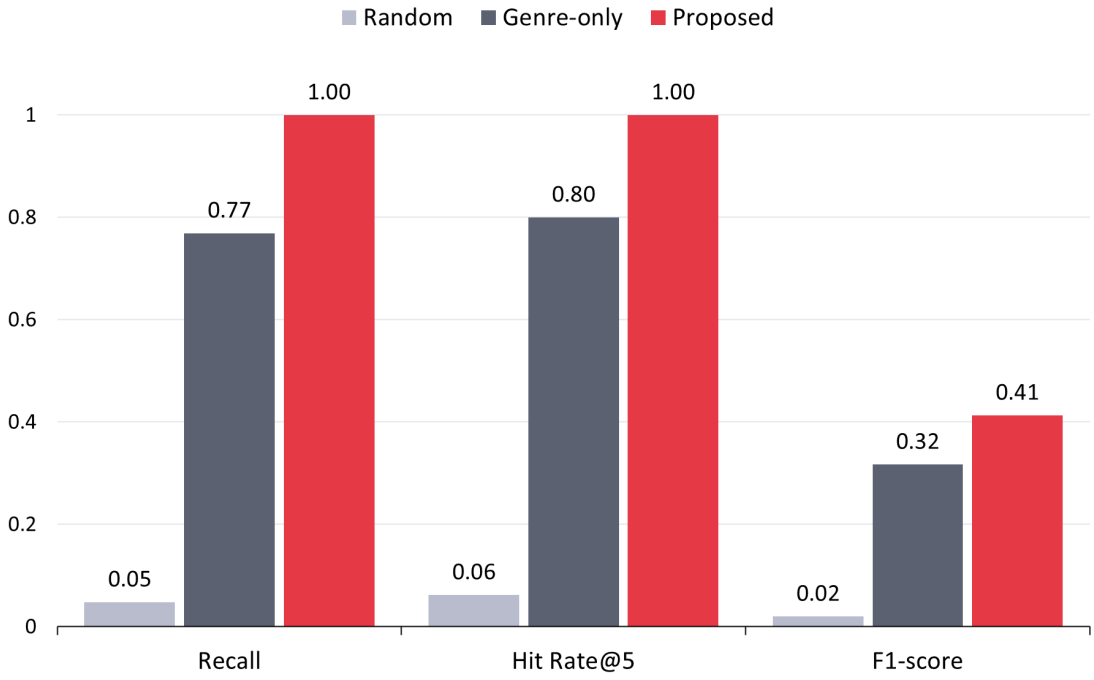


Figure 2: Recall, Hit Rate@5 and F1-score of the three models

6. RESULTS ANALYSIS & DISCUSSION

6.1 Confusion Matrix

	Predicted: in top 5	Predicted: not in top 5
Actual: expected	26	0
Actual: not expected	74	2,080

The matrix sums the proposed model's decisions over the 20 test queries, 2,180 query–movie decisions in total. All 26 expected movies were recommended (no false negatives), so recall is 1.00. The 74 false positives are the remaining top-five slots: because each query has only one or two expected answers, at least three of its five slots must hold other movies, and most of these are still sensible (for example Joker for The Dark Knight). Accuracy is 0.966 but is dominated by the 2,080 true negatives, so we rely on recall, F1 and hit rate instead. For comparison, the genre-only model missed 6 of the expected movies (TP = 20, FN = 6).

6.2 Predictions and Recommendation Quality

Figure 3 shows three example queries. For The Dark Knight the model returns the other two Batman films first (67% and 60%), then Joker (shares Gotham and the crime genre) and The Prestige (same director and lead actor). For Toy Story it returns Toy Story 3 (80%) and then other Pixar films, and for 3 Idiots it returns PK (same director and lead actor), followed by Dangal, Zindagi Na Milegi Dobara and Lagaan. A misspelt query such as "toy stroy" is still matched to Toy Story.

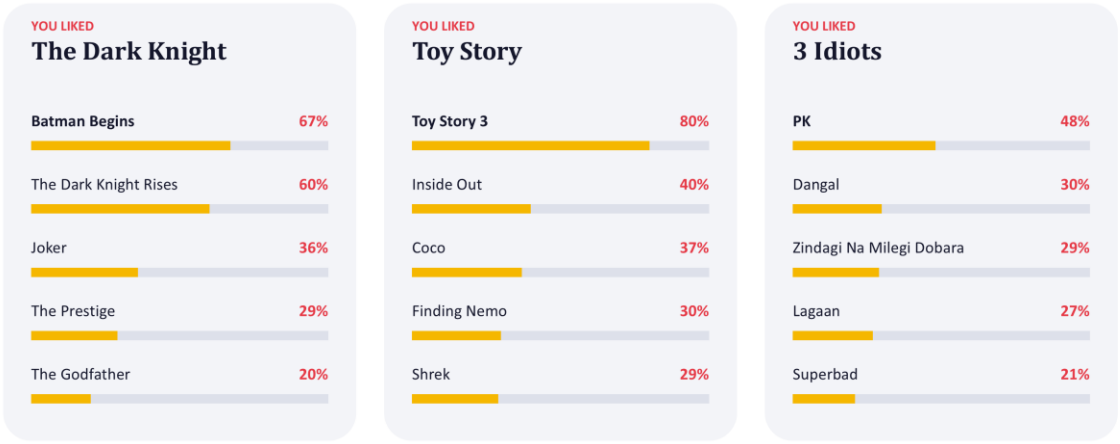


Figure 3: Top-5 recommendations for three example movies

The model struggles when similarity depends on ideas rather than shared words. For Inception it ranks The Dark Knight Rises and Mission: Impossible – Fallout first, mainly because of shared genres, the director and the keyword "heist", while The Matrix, arguably the closest film in theme, comes only fourth. The last places in a top-five list are also weaker, since they often share only genres (The Godfather for The Dark Knight, at 20%).

6.3 Comparison with Baseline

The proposed model raises recall from 0.769 (genre-only) to 1.000, finding 6 more expected movies, and F1 from 0.317 to 0.413; compared with random picks, recall is about 20 times higher. The reason is

visible in the features. A genre-only model gives exactly the same score to every movie with the same genre set, so The Dark Knight looks equally similar to any action-crime film and ties are broken arbitrarily. Adding the director, lead actors and keywords breaks these ties with information that actually identifies related films (christophernolan, christianbale, batman, gotham). Random selection picks each expected movie with a probability of only 5/109, which explains its recall of 0.048.

6.4 Observations

- Franchise and same-director movies consistently appear at the top of the list, with similarity scores between 0.48 and 0.80.
- The lower positions of the top five usually share only genres and are weaker suggestions, with similarity around 0.20–0.30.
- Accuracy barely separates the models (0.943 to 0.966) because almost all decisions are true negatives; recall, F1 and hit rate separate them far better.

6.5 Strengths

- Very simple: about 100 lines of Python, no training loop and no hyper-parameters to tune.
- Needs no user data, so it works for new users and for new movies as soon as their description is added (no cold-start problem).
- Explainable: every recommendation can be justified by the words the two movies share.

6.6 Limitations

- Quality depends entirely on the hand-written tags; a missing keyword means a missed connection.
- It recommends "more of the same" and cannot surprise the user or learn from what they actually watch or rate.
- The dataset (110 movies) and the test set (20 queries labelled by the team) are small, so the results show that the approach works but are not a production-level benchmark.

7. CONCLUSION, FUTURE SCOPE & REFERENCES

7.1 Conclusion

We built a movie recommendation system based on content-based filtering: each movie is described by a bag of words drawn from its genres, director, lead actors and plot keywords, and cosine similarity finds the five most similar movies. On a labelled test set of 20 queries the model found all 26 expected movies (recall 1.00, hit rate 1.00, F1 0.41), clearly ahead of a genre-only model (recall 0.77) and random picks (recall 0.05). The main takeaway is that a very simple, explainable model with no training loop can give useful recommendations when its features are chosen well.

7.2 Future Scope

1. Scale the system to larger public datasets such as TMDB 5000 or MovieLens [7], using nearest-neighbour search instead of a full similarity matrix.

2. Use TF-IDF weighting or sentence embeddings of full plot summaries to capture themes that single keywords miss.

3. Combine the model with collaborative filtering on user ratings to form a hybrid recommender, and deploy it as a Streamlit web app with movie posters.

7.3 Limitations Recap

As discussed in Section 6.6, the model only knows the tags we give it, cannot personalise from user behaviour, and was evaluated on a small, hand-labelled test set. The reported numbers therefore show that the approach works on this catalogue; they should not be read as the performance a large streaming service would see.

7.4 References

[1] F. Ricci, L. Rokach and B. Shapira, "Introduction to Recommender Systems Handbook", in Recommender Systems Handbook, Springer, 2011.

[2] M. J. Pazzani and D. Billsus, "Content-Based Recommendation Systems", in The Adaptive Web, LNCS 4321, Springer, 2007.

[3] C. D. Manning, P. Raghavan and H. Schütze, "Introduction to Information Retrieval", Cambridge University Press, 2008.

[4] Y. Koren, R. Bell and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems", IEEE Computer, vol. 42, no. 8, 2009.

[5] X. He, L. Liao, H. Zhang, L. Nie, X. Hu and T.-S. Chua, "Neural Collaborative Filtering", Proc. WWW, 2017.

[6] P. Covington, J. Adams and E. Sargin, "Deep Neural Networks for YouTube Recommendations", Proc. ACM RecSys, 2016.

[7] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context", ACM Transactions on Interactive Intelligent Systems, vol. 5, no. 4, 2015.

[8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] scikit-learn developers, "CountVectorizer" and "cosine_similarity" documentation, https://scikit-learn.org (accessed 2026).